Installer un système de Log via Graylog

Sommaire

Sommaire.....1
Informations additionnelles.....1
Partie 1 - Présentation de Graylog.....1
Partie 2 - Les prérequis.....2
Partie 3 - Installation de Graylog et de ses modules.....2
Sous partie 1 - Installation de MongoDB.....2
Sous partie 2 - Installation d'OpenSearch.....4
Sous partie 3 - Configuration de Java (JVM).....5
Sous partie 4 - Installation de Graylog.....7
Partie 4 - Création d'un utilisateur.....12

Informations additionnelles

Os utilisé →	Debian 12.10
Procédure réalisée sur machine virtuelle →	Oui, Hyper V
Texte écrit dans cette couleur →	Commande

Partie 1 - Présentation de Graylog

Graylog est une plateforme open-source de gestion et d'analyse des journaux (logs) informatiques, utilisée principalement pour la surveillance, le diagnostic et la sécurité des systèmes informatiques. Elle permet de centraliser les logs provenant de diverses sources (serveurs, applications, équipements réseau, etc.), de les indexer et de les analyser en

temps réel grâce à une interface web conviviale. Graylog repose sur Elasticsearch pour le stockage et la recherche rapide des données, et utilise MongoDB pour la gestion de la configuration. Grâce à ses capacités de filtrage, d'alerte et de visualisation, Graylog aide les administrateurs systèmes et les équipes de sécurité à détecter rapidement les anomalies, à résoudre les incidents et à améliorer la performance globale de leur infrastructure IT.

Partie 2 - Les prérequis

1) Une machine sous Debian 12 avec au minimum 8go de RAM et une IP statique. (voir la procédure)
2) Définir un serveur NTP pour la synchronisation de la date et de l'heure :
   → timedatectl set-timezone Europe/Paris
3) Vous pouvez paramétrer un accès en SSH sur la machine mais cela reste optionnel. (voir la procédure)

Partie 3 - Installation de Graylog et de ses modules

Nous allons commencer par mettre à jour le cache des paquets ainsi que l'installation d'outils nécessaire pour la suite.

→ apt update
→ apt-get install curl lsb-release ca-certificates gnupg2 pwgen

Sous partie 1 - Installation de MongoDB

Téléchargez la clé GPG correspondante au dépôt MongoDB :

→ curl -fsSL https://www.mongodb.org/static/pgp/server-6.0.asc | gpg -o /usr/share/keyrings/mongodb-server-6.0.gpg --dearmor

Il faudra ensuite l'ajouter sur la machine :

→ echo "deb [signed-by=/usr/share/keyrings/mongodb-server-6.0.gpg] http://repo.mongodb.org/apt/debian bullseye/mongodb-org/6.0 main" | tee /etc/apt/sources.list.d/mongodb-org-6.0.list

Une fois cela fait il faudra remettre à jour le cache des paquets et installer MongoDB

→ apt update
→ apt install -y mongodb-org

Une erreur va alors se produire lors de l'installation.

```
root@debian-graylog:~# apt install -y mongodb-org
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Certains paquets ne peuvent être installés. Ceci peut signifier
que vous avez demandé l'impossible, ou bien, si vous utilisez
la distribution unstable, que certains paquets n'ont pas encore
été créés ou ne sont pas sortis d'Incoming.
L'information suivante devrait vous aider à résoudre la situation :

Les paquets suivants contiennent des dépendances non satisfaites :
 mongodb-org-mongos : Dépend: libssl1.1 (>= 1.1.1) mais il n'est pas installable
 mongodb-org-server : Dépend: libssl1.1 (>= 1.1.1) mais il n'est pas installable
E: Impossible de corriger les problèmes, des paquets défectueux sont en mode « garder en l'état ».
```

Cela s'explique par le manque d'une dépendance que nous allons devoir installer manuellement.

→ wget http://archive.ubuntu.com/ubuntu/pool/main/o/openssl/libssl1.1_1.1.1f-1ubuntu2.24_amd64.deb
→ dpkg -i libssl1.1_1.1.1f-1ubuntu2.24_amd64.deb

On peut alors relancer l'installation de MongoDB en rappelant la commande ou en la re tapant :

→ apt install -y mongodb-org

Une fois l'installation terminée, nous allons devoir relancer le service MongoDB et activer son démarrage automatique au lancement de la machine Debian.

→ systemctl daemon-reload
→ systemctl enable mongod.service
→ systemctl restart mongod.service
→ systemctl --type=service --state=active | grep mongod

Afin de vérifier si notre service est bien lancé on peut faire :

→ systemctl status mongod.service

```
root@debian-graylog:~# systemctl status mongod.service
● mongod.service - MongoDB Database Server
     Loaded: loaded (/lib/systemd/system/mongod.service; enabled; preset: enabled)
     Active: active (running) since Wed 2025-06-11 15:13:38 CEST; 1min 45s ago
       Docs: https://docs.mongodb.org/manual
   Main PID: 1821 (mongod)
     Memory: 81.4M
        CPU: 1.835s
     CGroup: /system.slice/mongod.service
             └─1821 /usr/bin/mongod --config /etc/mongod.conf
```

Sous partie 2 - Installation d'OpenSearch

Nous allons à présent installer OpenSearch

Commençons par l'ajout de la clé de signature pour les paquets d'OpenSearch :

→ curl -o- https://artifacts.opensearch.org/publickeys/opensearch.pgp | gpg --dearmor --batch --yes -o /usr/share/keyrings/opensearch-keyring

Il faudra ensuite ajouter le dépôt OpenSearch afin de télécharger le paquet :

→ echo "deb [signed-by=/usr/share/keyrings/opensearch-keyring] https://artifacts.opensearch.org/releases/bundle/opensearch/2.x/apt stable main" | tee /etc/apt/sources.list.d/opensearch-2.x.list

Une fois cela fait, mettre à jour le cache de paquets :

→ apt update

Nous allons installer OpenSearch en définissant le mot de passe par défaut du compte Administrateur. Dans notre cas, il s'agira de "Armel1234!l". Il faudra le remplacer par la suite avec un mot de passe fort.

→ env OPENSEARCH_INITIAL_ADMIN_PASSWORD=Armel1234! apt-get install opensearch

Une fois l'installation terminée, nous allons ouvrir le fichier de configuration afin d'effectuer la configuration minimale.

→ nano /etc/opensearch/opensearch.yml

A l'intérieur de ce fichier, ajoutez les lignes suivantes :

→ cluster.name: graylog
→ node.name: ${HOSTNAME}
→ path.data: /var/lib/opensearch
→ path.logs: /var/log/opensearch
→ discovery.type: single-node
→ network.host: 127.0.0.1
→ action.auto_create_index: false
→ plugins.security.disabled: true

⚠️Certaines lignes sont déjà présentes mais commentées. Vous pouvez les décommenter ou alors ajouter l'ensemble à la fin du fichier.

Enregistrez et fermez ce fichier.

graylog

Sous partie 3 - Configuration de Java (JVM)

Il va falloir configurer Java Virtual Machine utilisé par OpenSearch afin d'ajuster la quantitée de mémoire utilisé par le service :

→ nano /etc/opensearch/jvm.options

Nous avons alloué 8go à notre machine, et sur ces 8go nous voulons attribuer que 4go pour OpenSearch. Il faudra alors changer les paramètres suivant :

```
  GNU nano 7.2
## JVM configuration

################################################################
## IMPORTANT: JVM heap size
################################################################
##
## You should always set the min and max JVM heap
## size to the same value. For example, to set
## the heap to 4 GB, set:
##
## -Xms4g
## -Xmx4g
##
## See https://opensearch.org/docs/opensearch/install/important-settings/
## for more information
##
################################################################

# Xms represents the initial size of total heap space
# Xmx represents the maximum size of total heap space

-Xms1g
-Xmx1g
```

Par ceux-ci :

```
  GNU nano 7.2
## JVM configuration

################################################################
## IMPORTANT: JVM heap size
################################################################
##
## You should always set the min and max JVM heap
## size to the same value. For example, to set
## the heap to 4 GB, set:
##
## -Xms4g
## -Xmx4g
##
## See https://opensearch.org/docs/opensearch/install/important-settings/
## for more information
##
################################################################

# Xms represents the initial size of total heap space
# Xmx represents the maximum size of total heap space

-Xms4g
-Xmx4g
```

Il faudra ensuite enregistrer et fermer le fichier.

Nous allons également vérifier la configuration du paramètre “max_map_count” qui définit la limite des zones de mémoire mappés par processus. Nous allons fixer cette valeur à 262144 pour éviter les erreurs liées à la gestion de la mémoire.

→ sysctl -w vm.max_map_count=262144

Nous allons ensuite terminer par désactiver le démarrage automatique d’OpenSearch et lancer le service associé.

→ systemctl daemon-reload
→ systemctl enable opensearch
→ systemctl restart opensearch

Il n’est pas impossible qu'à cette étape vous rencontreriez une erreur vous indiquant que vous n’avez pas assez de mémoire pour Java. Il faudra aller dans les paramètres de votre hyperviseur, et désactiver la mémoire dynamique.

Votre OpenSearch aura correctement démarré :

```
root@debian-graylog:~# sudo systemctl status opensearch
● opensearch.service - OpenSearch
     Loaded: loaded (/lib/systemd/system/opensearch.service; enabled; preset: enabled)
     Active: active (running) since Wed 2025-06-11 16:21:25 CEST; 1min 53s ago
       Docs: https://opensearch.org/
   Main PID: 3263 (java)
      Tasks: 70 (limit: 9468)
     Memory: 4.3G
        CPU: 1min 4.375s
     CGroup: /system.slice/opensearch.service
             └─3263 /usr/share/opensearch/jdk/bin/java -Xshare:auto -Dopensearch.networ
```

Vous allez pouvoir vérifier que java est bien été créée et comporte 4go de ram via la commande suivante :

→ top

```
top - 16:24:54 up  1:27,  2 users,  load average: 0,07, 0,20, 0,09
Tâches: 105 total,   1 en cours, 104 en veille,   0 arrêté,   0 zombie
%Cpu(s):  0,0 ut,  0,0 sy,  0,0 ni,100,0 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
MiB Mem :   7940,1 total,    185,5 libr,   5016,3 util,   3007,9 tamp/cache
MiB Éch :    977,0 total,    9[illegible]7 libr,      0,3 util.   2923,8 dispo Mem

    PID UTIL.     PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TEMPS+ COM.
   2552 root      20   0       0      0      0 I   0,0   0,0   0:00.07 kworker/0:0-events
   2567 root      20   0       0      0      0 I   0,0   0,0   0:00.02 kworker/2:1-mm_percpu_wq
   2570 root      20   0       0      0      0 I   0,[illegible]   0,0   0:00.21 kworker/u8:1-events_unbound
   2728 root      20   0       0      0      0 I   0,0   0,[illegible]   0:00.11 kworker/u8:0-flush-8:0
   2730 root      20   0       0      0      0 I   0,0   0,0   [illegible]02 kworker/2:0-events
   2732 root      20   0       0      0      0 I   0,0   0,0   0:[illegible] kworker/u8:2-events_unbound
   3263 opensea+  20   0 8107824   4,4g  26660 S   0,0  56,8   1:02.90 java
   3498 root      20   0       0      0      0 I   0,0   0,0   0:00.01 kworker/u8:3-events_unbound
   3499 root      20   0       0      0      0 I   0,0   0,0   0:00.00 kworker/u8:4
   3500 root      20   0       0      0      0 I   0,0   0,0   0:00.04 kworker/u8:5-events_unbound
   3503 root      20   0       0      0      0 I   0,0   0,0   0:00.00 kworker/2:2-mm_percpu_wq
   3511 root      20   0   11708   5228   3304 R   0,0   0,1   0:01.65 top
```

Nous allons pouvoir passer à la dernière étape, l'installation de Graylog

Sous partie 4 - Installation de Graylog

Nous allons télécharger Graylog dans sa version 6.1

→ wget https://packages.graylog2.org/repo/packages/graylog-6.1-repository_latest.deb
→ dpkg -i graylog-6.1-repository_latest.deb
→ apt update
→ apt install graylog-server

Avant de vouloir lancer Graylog, nous avons deux options à configurer.

- Le "passsword_secret" : C'est ce mot de passe qui est utilisé par Graylog pour sécuriser le stockage des mots de passe des utilisateurs. Elle doit être unique et aléatoire
- Le "root_password_sha2" : C'est le mot de passe par défaut de l'administrateur dans Graylog. Il est stocké sous forme d'un hash SHA-256

Pour le “password_secret” nous allons générer une clé de 96 caractères :

→ pwgen -N 1 -s 96

Copiez le résultat puis ouvrez le fichier suivant :

→ nano /etc/graylog/server/server.conf

On va venir coller la clé à l’endroit prévu dans le fichier

```
  GNU nano 7.2                                                                    /etc/graylog/server/server.conf *
# * The characters newline, carriage return, and tab can be inserted with characters \n, \r, and \t, respectively.
#
# * The backslash character must be escaped as a double backslash. For example:
#
# path=c:\\docs\\doc1
#

# If you are running more than one instances of Graylog server you have to select one of these
# instances as leader. The leader will perform some periodical tasks that non-leaders won't perform.
is_leader = true

# The auto-generated node ID will be stored in this file and read after restarts. It is a good idea
# to use an absolute file path here if you are starting Graylog server from init scripts or similar.
node_id_file = /etc/graylog/server/node-id

# You MUST set a secret to secure/pepper the stored user passwords here. Use at least 64 characters.
# Generate one by using for example: pwgen -N 1 -s 96
# ATTENTION: This value must be the same on all Graylog nodes in the cluster.
# Changing this value after installation will invalidate all user sessions and encrypted values in the database invalid. (e.g. encrypted access tokens)
password_secret = aKPRVO4UhWPQwH9Bt9EI6EY2VAG9T555a9odoqqZXfxlANksvcGG8MI5ckrTHT1uSouD41GpwLmecUOlCjOQIPINJAWaSriE

# The default root user is named 'admin'
#root_username = admin
```

Enregistrez et fermez le fichier.

Pour le second mot de passe (celui du compte admin par défaut), nous allons prendre le mot de passe “Armel1234!”

→ echo -n "Armel1234!" | shasum -a 256

Copiez la valeur retourné puis retirez le “-” à la fin.

Le resultat est celui ci :
c8d9a99ca3ca87568a8581cb7e0d6c8febd9c030f35bc07651fb0f480a6a5aa9

On retourne dans le fichier de configuration Graylog puis on va venir coller le mot de passe hashé à l’endroit prévu :

```
  GNU nano 7.2                                                                    /etc/graylog/server/server.conf *
#   property value is not ignored, and is treated as part of the property value.
#
# * A property value can span several lines if each line is terminated by a backslash ('\') character.
#   For example:
#
# targetCities=\
#         Detroit,\
#         Chicago,\
#         Los Angeles
#
#   This is equivalent to targetCities=Detroit,Chicago,Los Angeles (white space at the beginning of lines is ignored).
#
# * The characters newline, carriage return, and tab can be inserted with characters \n, \r, and \t, respectively.
#
# * The backslash character must be escaped as a double backslash. For example:
#
# path=c:\\docs\\doc1
#

# If you are running more than one instances of Graylog server you have to select one of these
# instances as leader. The leader will perform some periodical tasks that non-leaders won't perform.
is_leader = true

# The auto-generated node ID will be stored in this file and read after restarts. It is a good idea
# to use an absolute file path here if you are starting Graylog server from init scripts or similar.
node_id_file = /etc/graylog/server/node-id

# You MUST set a secret to secure/pepper the stored user passwords here. Use at least 64 characters.
# Generate one by using for example: pwgen -N 1 -s 96
# ATTENTION: This value must be the same on all Graylog nodes in the cluster.
# Changing this value after installation will render all user sessions and encrypted values in the database invalid. (e.g. encrypted access tokens)
password_secret = aKPRVO4UhWPQwH9Bt9EI6EY2VAG9T555a9odoqqZXfx1ANksvcGG8MI5ckrTHT1uSouD41GpwLmecU01CjOQIPINJAWaSriE

# The default root user is named 'admin'
#root_username = admin

# You MUST specify a hash password for the root user (which you only need to initially set up the
# system and in case you lose connectivity to your authentication backend)
# This password cannot be changed using the API or via the web interface. If you need to change it,
# modify it in this file.
# Create one by using for example: echo -n yourpassword | shasum -a 256
# and put the resulting hash value into the following line
root_password_sha2 = 13004d8331d779808a2336d46b3553d1594229e2bb696a8e9e14554d82a648da
```

Profitez d'être dans ce fichier pour terminer la configuration :

1) Nous allons indiquer que l'interface web de Graylog soit accessible sur le pour 9000 via n'importe quel ip du serveur. La ligne est donc :
   http_bind_address = 0.0.0.0:9000

```
  GNU nano 7.2                                                                    /etc/g

#### HTTP bind address
#
# The network interface used by the Graylog HTTP interface.
#
# This network interface must be accessible by all Graylog nodes in the cluster and by all clients
# using the Graylog web interface.
#
# If the port is omitted, Graylog will use port 9000 by default.
#
# Default: 127.0.0.1:9000
http_bind_address = 0.0.0.0:9000
#http_bind_address = [2001:db8::1]:9000

#### HTTP publish URI
#
# The HTTP URI of this Graylog node which is used to communicate with the other Graylog nodes in the cluster an
# clients using the Graylog web interface.
#
# The URI will be published in the cluster discovery APIs, so that other Graylog nodes will be able to find and
#
# This configuration setting has to be used if this Graylog node is available on another network interface than
# for example if the machine has multiple network interfaces or is behind a NAT gateway.
#
# If $http_bind_address contains a wildcard IPv4 address (0.0.0.0), the first non-loopback IPv4 address of this
# This configuration setting *must not* contain a wildcard address!
#
# Default: http://$http_bind_address/
#http_publish_uri = http://192.168.1.1:9000/

#### External Graylog URI
#
# The public URI of Graylog which will be used by the Graylog web interface to communicate with the Graylog RES
#
# The external Graylog URI usually has to be specified, if Graylog is running behind a reverse proxy or load-ba
# and it will be used to generate URLs addressing entities in the Graylog REST API (see $http_bind_address).
#
# When using Graylog Collector, this URI will be used to receive heartbeat messages and must be accessible for
#
```

2) Nous allons configurer l'option "elasticsearch_hosts" qui permet de déclarer notre instance locale OpenSearch. Nous allons mettre la ligne suivante : elasticsearch_hosts = http://127.0.0.1:9200

```
  GNU nano 7.2                                                    /etc/graylog/server/server.conf *
#http_enable_gzip = false

# The maximum size of the HTTP request headers in bytes.
#http_max_header_size = 8192

# The size of the thread pool used exclusively for serving the HTTP interface.
#http_thread_pool_size = 64

################
# HTTPS settings
################

#### Enable HTTPS support for the HTTP interface
#
# This secures the communication with the HTTP interface with TLS to prevent request forgery and eavesdropping.
#
# Default: false
#http_enable_tls = true

# The X.509 certificate chain file in PEM format to use for securing the HTTP interface.
#http_tls_cert_file = /path/to/graylog.crt

# The PKCS#8 private key file in PEM format to use for securing the HTTP interface.
#http_tls_key_file = /path/to/graylog.key

# The password to unlock the private key used for securing the HTTP interface.
#http_tls_key_password = secret

# If set to "true", Graylog will periodically investigate indices to figure out which fields are used in which streams.
# It will make field list in Graylog interface show only fields used in selected streams, but can decrease system performance,
# especially on systems with great number of streams and fields.
stream_aware_field_types=false

# Comma separated list of trusted proxies that are allowed to set the client address with X-Forwarded-For
# header. May be subnets, or hosts.
#trusted_proxies = 127.0.0.1/32, 0:0:0:0:0:0:0:1/128

# List of Elasticsearch hosts Graylog should connect to.
# Need to be specified as a comma-separated list of valid URIs for the http ports of your elasticsearch nodes.
# If one or more of your elasticsearch hosts require authentication, include the credentials in each node URI that
# requires authentication.
#
# Default: http://127.0.0.1:9200
elasticsearch_hosts = http://127.0.0.1:9200

# Maximum number of attempts to connect to elasticsearch on boot for the version probe.
#
# Default: 0, retry indefinitely with the given delay until a connection could be established
#elasticsearch_version_probe_attempts = 5

# Waiting time in between connection attempts for elasticsearch_version_probe_attempts
#
# Default: 5s
#elasticsearch_version_probe_delay = 5s

# Maximum amount of time to wait for successful connection to Elasticsearch HTTP port.
#
```

Une fois cela fait, enregistrez et fermez le fichier.

Nous allons activer Graylog pour qu'il démarre seul au prochaine démarrage de la machine

→ systemctl enable --now graylog-server

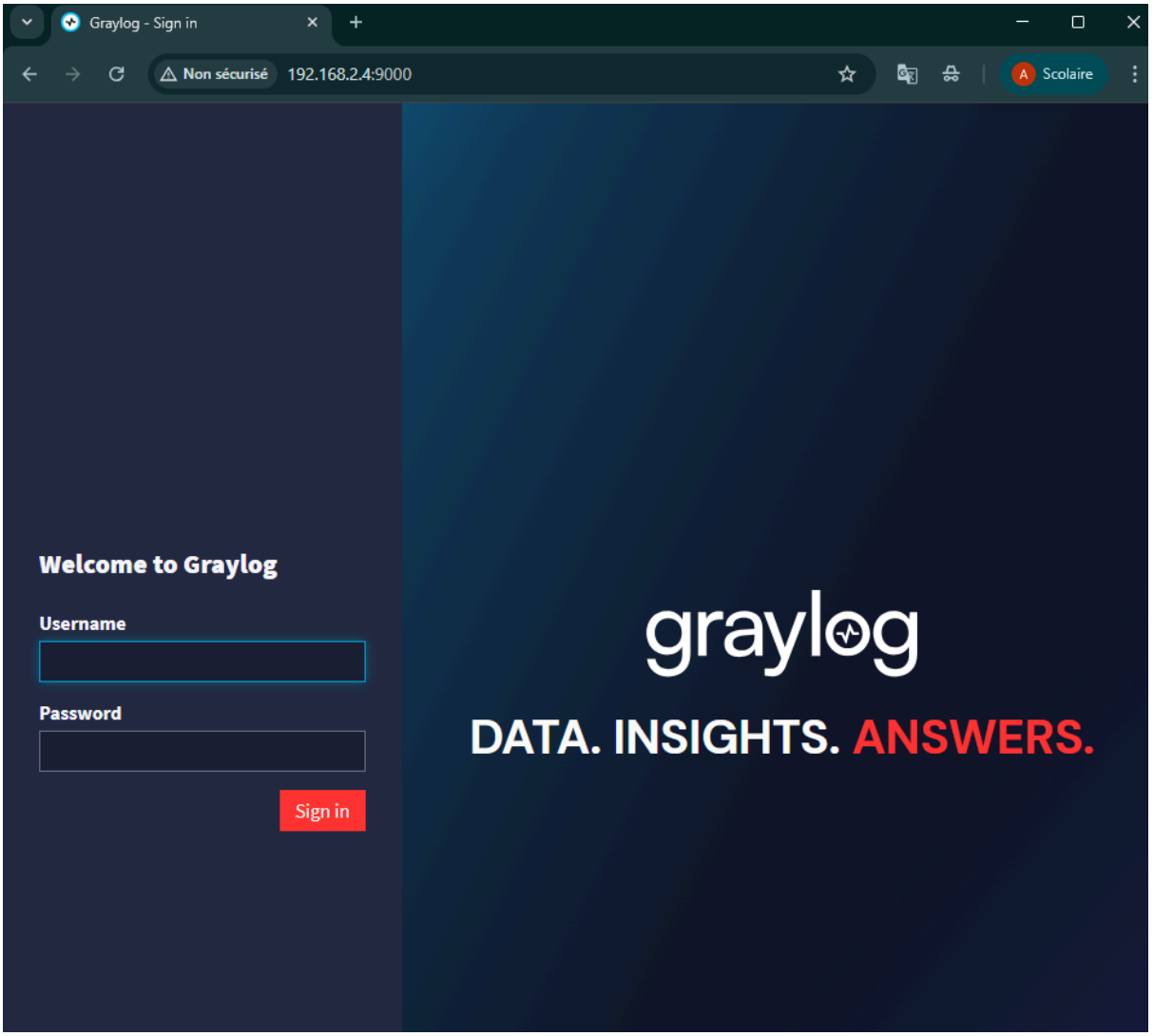
Avant la connexion à l'interface web, on peut vérifier que le service a bien démarré.

→ systemctl status graylog-server.service

```
root@debian-graylog:~# systemctl status graylog-server.service
● graylog-server.service - Graylog server
     Loaded: loaded (/lib/systemd/system/graylog-server.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-06-12 10:14:00 CEST; 814ms ago
       Docs: http://docs.graylog.org/
   Main PID: 2196 (graylog-server)
      Tasks: 21 (limit: 9468)
     Memory: 91.2M
        CPU: 1.719s
```

Nous allons alors pouvoir entrer l'adresse IP de notre serveur dans un navigateur pour nous connecter à l'interface graphique de Graylog. Il ne faudra pas oublier le port.

→ 192.168.2.4:9000



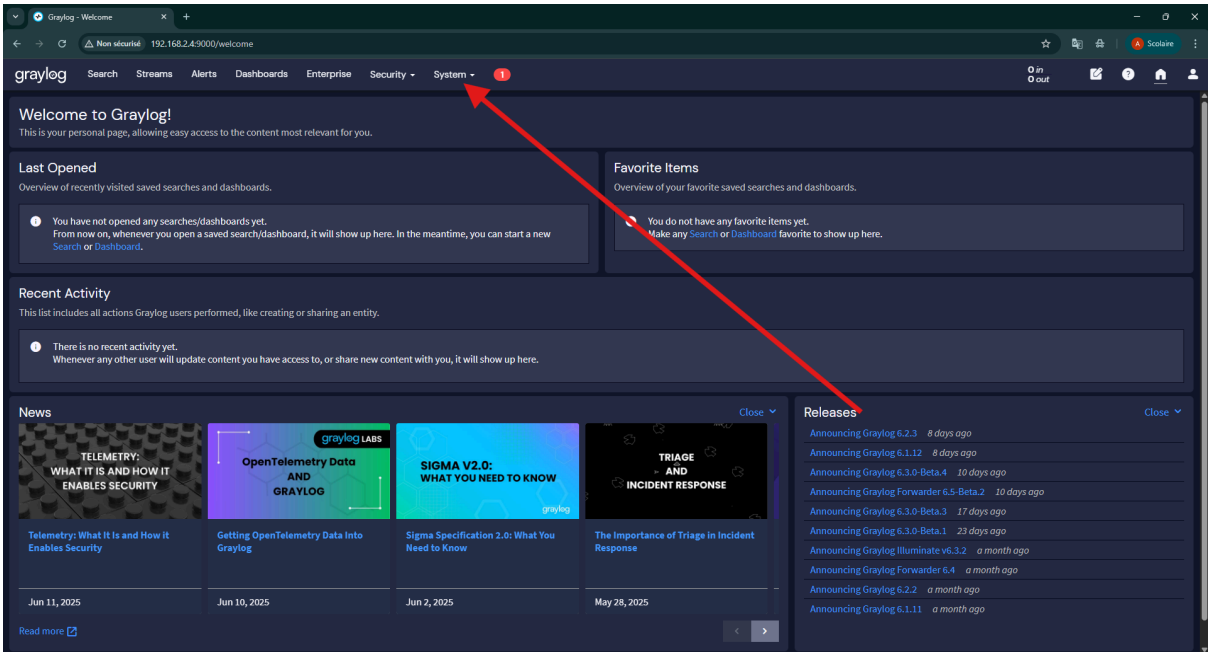
Pour se connecter les identifiants sont :

Username : admin (par défaut)
Password : Armel1234! (c'est celui que nous avons configuré précédemment)

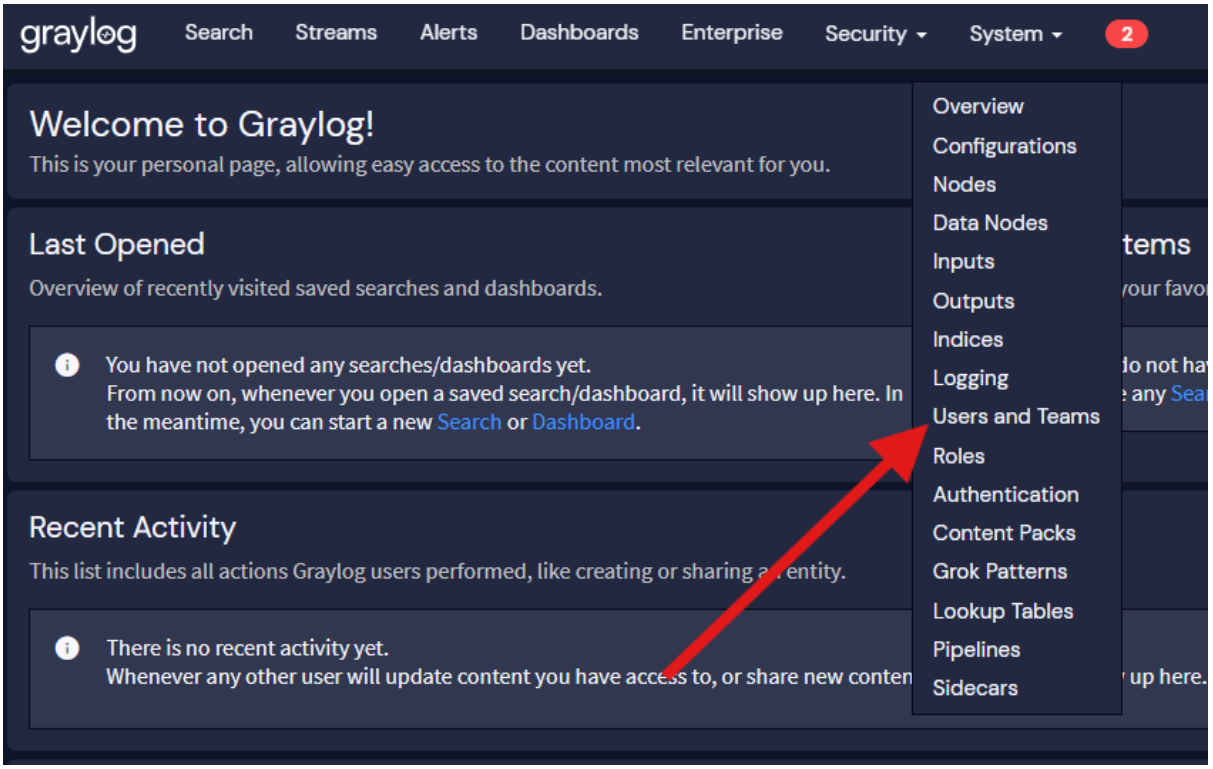
Partie 4 - Création d’un utilisateur

Vous êtes désormais connecté à l’interface web de Graylog. Cependant pour plus de sécurité, nous voulons créer un compte utilisateur pour ne pas utiliser celui par défaut.

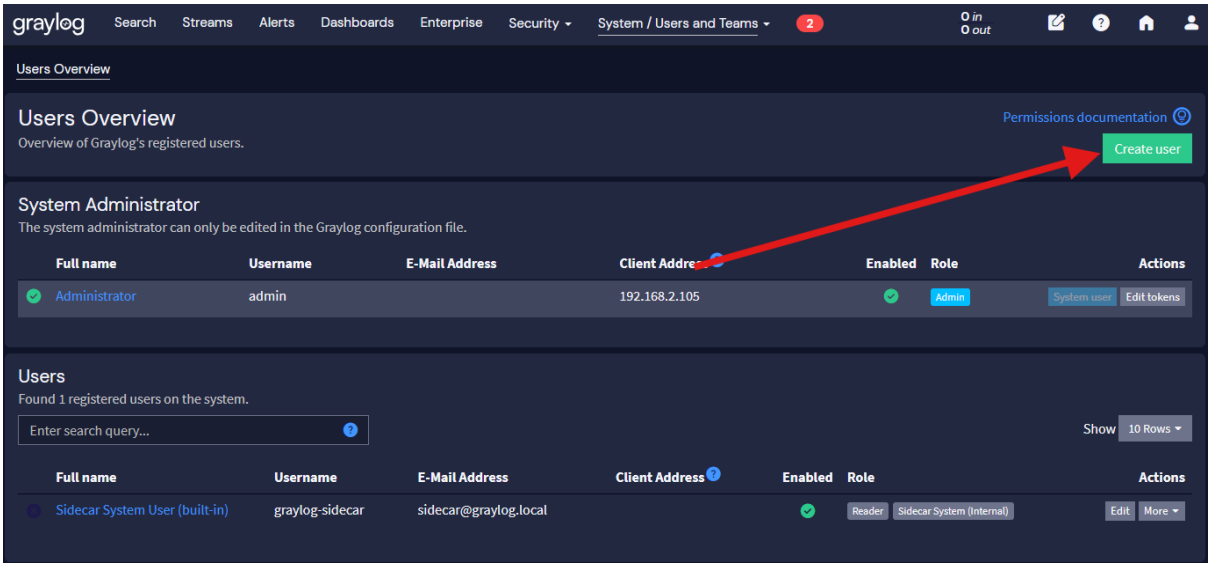
Il faudra se rendre dans système



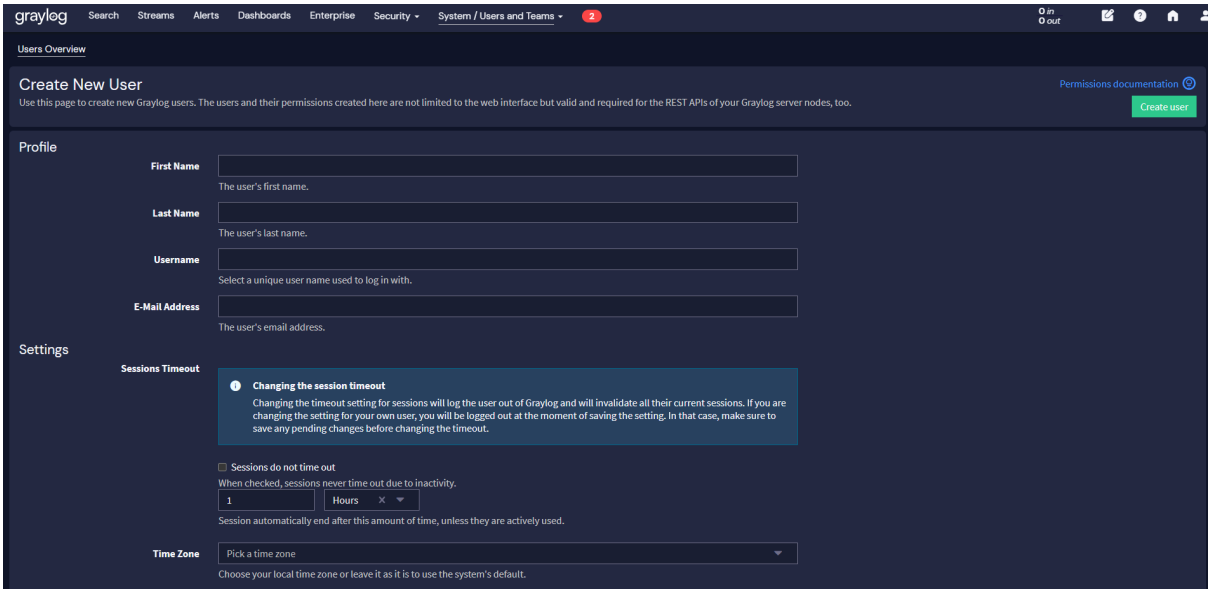
Puis Users and Teams



Une fois dans le bon menu, nous allons faire “create user”



Vous devrez alors compléter le formulaire et votre compte sera créée



Félicitations, vous avez installé Graylog, il faudra maintenant le configurer afin d'obtenir des logs. Chose que vous verrez dans une prochaine procédure.